

API Guidelines

Version: 1.0

Purpose

This document defines the API and interface design guidelines for Trading Platform Pro and its primary application, the Trading Cockpit.

The guidelines apply to explicit software boundaries used by Domain, Application, Infrastructure and Presentation components.

APIs shall support concrete product workflows and architecture boundaries.

Avoid introducing public or extensible APIs for hypothetical future requirements.

Scope

These guidelines apply to:

- Internal Python interfaces
- Application ports
- Repository interfaces
- Application service interfaces
- Infrastructure adapter interfaces
- Broker integration interfaces
- Market data interfaces
- Presentation-facing application interfaces

Future interfaces such as:

- Plugin APIs
- REST APIs
- WebSocket APIs

require a concrete product requirement and explicit architecture review before introduction.

API Design Principles

Every API shall be:

- explicit
- minimal
- predictable
- strongly typed
- responsibility-oriented
- testable
- documented where externally consumed
- aligned with architecture boundaries

Prefer narrow interfaces that represent one capability.

Avoid generic interfaces that expose unrelated operations.

Product and Capability Alignment

An API shall support a defined:

- product workflow
- application use case
- domain capability
- technical integration boundary

Before introducing an interface identify:

1. The consumer.
2. The owner.
3. The capability boundary.

4. The required operations.
 5. The expected failure behaviour.
- Do not introduce interfaces only to anticipate future reuse.

Architecture Boundary Rules

Interfaces shall preserve the architecture dependency direction.

```
```text
```

Presentation

↓

Application

↓

Domain

Infrastructure

↓

Application / Domain Ports

```
```
```

Examples:

- Presentation consumes Application services.
 - Application consumes repository or integration ports.
 - Infrastructure implements ports.
 - Domain remains independent from technical APIs.
- Presentation shall not consume concrete Infrastructure adapters directly.
Domain APIs shall not expose Infrastructure types.

Ports and Adapters

External technical capabilities shall be represented through explicit ports where application workflows require abstraction.

Example:

```
```text
```

Application

■

▼

BrokerPort

■

▼

InteractiveBrokersAdapter

```
```
```

The port defines the application-oriented contract.

The adapter implements provider-specific behaviour.

Ports shall use application-oriented or domain-oriented language.

Adapters may use provider-specific language internally.

Port Design

A port shall define the minimum capability required by its consumer.

Example:

```
```python
```

```
class BrokerOrderPort(Protocol):
```

```
 async def submit_order(
```

```
 self,
```

```
 request: SubmitOrderRequest,
```

```
) -> BrokerOrderSubmission:
```

```
 ...
```

```
```
```

Avoid large interfaces such as:

```
```python
class BrokerService:
 ...
```
```

when unrelated capabilities can be separated.

Potential broker capabilities may include:

- BrokerConnectionPort
- BrokerAccountPort
- BrokerPortfolioPort
- BrokerOrderPort
- BrokerExecutionPort

Interface separation shall be based on real consumers and workflows.

Do not create all potential ports before they are required.

Adapter Design

Infrastructure adapters implement explicit ports.

Adapters are responsible for:

- provider communication
- provider identifier translation
- provider model translation
- provider error translation
- transport lifecycle
- technical connection state

Adapters shall not contain:

- trading decisions
- candidate acceptance rules
- portfolio risk decisions
- order submission decisions
- position close decisions

Adapters translate and execute technical operations.

Naming

Names shall communicate capability and responsibility.

Classes:

```
```python
OrderApplicationService
TradingCandidateService
InteractiveBrokersOrderAdapter
```
```

Interfaces and ports:

```
```python
OrderRepository
BrokerOrderPort
MarketDataSubscriptionPort
```
```

Methods:

```
```python
get_order()
save_order()
submit_order()
cancel_order()
subscribe_quotes()
```
```

Avoid vague names such as:

```
```python
Manager
Handler
Processor
Helper
Utils
```
```

unless the specific responsibility is clearly defined by the complete name.
Avoid unnecessary abbreviations.

Method Design

Methods shall represent one clear operation.

Prefer:

```
```python
async def submit_order(
 request: SubmitOrderRequest,
) -> BrokerOrderSubmission:
 ...
```
```

Avoid:

```
```python
async def process(
 data: dict,
 mode: str,
 options: dict,
) -> object:
 ...
```
```

Method behaviour shall remain understandable from:

- name
- parameter types
- return type
- documented failure behaviour

Parameters

Parameters shall be explicit and strongly typed.

Prefer:

```
```python
create_order(command: CreateOrderCommand) -> OrderId
```
```

Avoid long parameter lists.

Use:

- Value Objects
- Commands
- Queries
- DTOs
- Request Models

where they clarify interface boundaries.

Do not use generic dictionaries as primary API contracts when a stable structure is known.

Avoid:

```
```python
submit_order(data: dict) -> dict
```
```

Return Values

Return values shall communicate meaningful operation results.

Prefer:

```
```python
OrderId
Order
BrokerOrderSubmission
QuoteSubscription
ReconciliationResult
```
```

Avoid ambiguous primitive results such as:

```
```python
True
False
None
```
```

when richer operation state is required.

A successful method return shall not imply external acknowledgement unless the returned type explicitly represents acknowledgement.

Domain Types

Domain interfaces shall use Domain types.

Examples:

- InstrumentId
- OrderId
- PositionId
- Money
- Price
- Quantity

Domain types shall not contain:

- SQLAlchemy models
- broker models
- provider payloads
- PySide6 types
- transport schemas

Domain APIs shall remain technology-independent.

Application DTOs

Application DTOs may be used to transfer structured data across application boundaries.

DTOs shall:

- have explicit fields
- be strongly typed
- remain free of provider-specific transport behaviour
- represent application requirements

Example:

```
```python
@dataclass(frozen=True)
class SubmitOrderRequest:
 order_id: OrderId
 instrument_id: InstrumentId
 action: OrderAction
 quantity: Quantity
```
```

```
order_type: OrderType
limit_price: Price | None
...
```

DTOs are not automatically Domain Entities.

Provider Models

Provider-specific models belong to Infrastructure.

Examples:

```
```python
lbContract
lbOrder
ProviderQuoteMessage
```
```

Provider models shall not cross into Domain APIs.

Adapters shall translate provider models into application-oriented or domain-oriented structures. Avoid leaking vendor identifiers into business interfaces unless the integration workflow explicitly requires them.

Persistence Models

Persistence models belong to Infrastructure.

Examples:

```
```python
OrderRecord
PositionRecord
SqlAlchemyOrderModel
```
```

Repository interfaces shall not expose SQLAlchemy models.

Prefer:

```
```python
def get(self, order_id: OrderId) -> Order | None:
 ...
 ...
```

Avoid:

```
```python
def get(self, order_id: str) -> SqlAlchemyOrderModel | None:
    ...
    ...
```

Async Interfaces

Interfaces shall be asynchronous when the underlying operation is inherently asynchronous or externally coordinated.

Typical asynchronous operations include:

- broker communication
- market data subscriptions
- external API calls
- asynchronous messaging
- long-running runtime operations

Example:

```
```python
async def submit_order(
 request: SubmitOrderRequest,
) -> BrokerOrderSubmission:
 ...
```

---

Do not mark interfaces asynchronous without an actual asynchronous boundary.  
Async behaviour shall remain explicit to callers.

---

## Cancellation

Long-running asynchronous interfaces shall support controlled cancellation where technically appropriate.

Cancellation shall:

- release owned resources
- preserve valid local state
- avoid uncontrolled external side effects
- remain distinguishable from external rejection

Cancellation behaviour shall be tested for business-critical operations.

---

## Timeout Behaviour

External operations shall have defined timeout behaviour where technically appropriate.

Examples:

- broker requests
- market data requests
- external APIs

Timeouts shall remain distinguishable from:

- explicit rejection
- invalid input
- disconnected state
- unavailable capability

Timeout behaviour shall be documented for critical interfaces.

---

## Exceptions

APIs shall fail explicitly.

Expected failures shall use meaningful exception types or explicit result states.

Examples:

```
```python
BrokerDisconnectedError
BrokerOrderRejectedError
MarketDataUnavailableError
RepositoryError
```
```

Domain rule violations shall use Domain Exceptions.

Infrastructure failures shall not be disguised as Domain Exceptions.

Avoid silently returning empty data after technical failure.

---

## Error Translation

Adapters shall translate provider-specific errors at the integration boundary.

Example:

```
```text
IB API Error
↓
InteractiveBrokersAdapter
↓
BrokerOrderRejectedError
```
```

Provider error context may be preserved for logging and diagnostics.  
Provider-specific error types shall not leak through application-facing ports unless explicitly required.

---

## Order Submission Interfaces

Order submission is business-critical.  
An order submission interface shall clearly distinguish:

- request accepted locally
- request transmitted
- broker acknowledgement
- broker rejection
- execution state

A successful API call shall not automatically mean that the broker acknowledged the order.  
Return types and state models shall preserve this distinction.

---

## Duplicate Submission Risk

Order APIs shall consider duplicate execution risk.  
Order submission interfaces should support an explicit application-controlled submission identity where required.

Example:

```
```python
@dataclass(frozen=True)
class SubmitOrderRequest:
    order_id: OrderId
    submission_key: OrderSubmissionKey
```

...

The application workflow owns duplicate-prevention rules.
Infrastructure may preserve and transport submission identifiers where technically possible.
Adapters shall not blindly retry order submission.

Idempotency

Idempotency requirements shall be evaluated per operation.
Typical candidates:

- read operations
- persistence updates with explicit identity
- subscription cancellation
- reconciliation reads

Order submission shall not be assumed idempotent.

An API shall document idempotency expectations where repeated execution may create external side effects.

Retry Behaviour

Retry behaviour shall not be hidden inside APIs without explicit design.
Retries shall consider:

- idempotency
- external side effects
- duplicate execution risk
- provider behaviour
- workflow ownership

Automatic retry may be appropriate for selected read operations.

Automatic order submission retry is prohibited unless explicitly defined and approved by the order workflow.

Retry attempts shall be observable.

Broker Interfaces

Broker interfaces may expose capabilities for:

- connection
- account information
- portfolio information
- positions
- order submission
- order cancellation
- order state
- executions

Broker interfaces shall expose operational state explicitly.

Example:

```
```python
async def get_connection_state(
 self,
) -> BrokerConnectionState:
 ...
```

The application shall not infer broker connectivity from unrelated successful operations.

---

## Market Data Interfaces

Market data interfaces may expose:

- instrument lookup
- quote retrieval
- quote subscription
- subscription cancellation
- connection state

Example:

```
```python
async def subscribe_quotes(
    instrument_id: InstrumentId,
) -> QuoteSubscription:
    ...
```

Market data contracts shall preserve:

- source timestamp where available
- availability state
- subscription state

Cached data shall not silently appear as current provider data.

Subscription Interfaces

Subscriptions shall have explicit lifecycle semantics.

Example:

```
```python
subscription = await market_data.subscribe_quotes(
 instrument_id,
)
await subscription.close()
```
```

Subscription interfaces shall define:

- ownership
- activation state
- cancellation
- disposal
- failure behaviour

Subscriptions shall not depend on garbage collection for critical resource release.

Repository Interfaces

Repository interfaces represent persistence requirements of Domain or Application workflows.

Example:

```
```python
class OrderRepository(Protocol):
 def get(
 self,
 order_id: OrderId,
) -> Order | None:
 ...
 def save(
 self,
 order: Order,
) -> None:
 ...
```
```

Repository interfaces shall:

- use domain-oriented identities
- expose meaningful persistence operations
- avoid SQLAlchemy types
- avoid database-specific query language

Do not create generic repositories solely to reduce repeated method names.

Query Interfaces

Read-oriented interfaces may return application-specific read models.

Example:

```
```python
@dataclass(frozen=True)
class OpenOrderView:
 order_id: OrderId
 instrument_id: InstrumentId
 status: OrderStatus
```
```

Read models shall remain distinct from mutable Domain Entities where their responsibility differs.
Queries shall not silently mutate business state.

Presentation-Facing Interfaces

Presentation shall interact with Application interfaces.

Presentation-facing APIs shall provide:

- explicit operation state
- UI-relevant read models
- observable loading or failure state where required

Presentation shall not receive:

- SQLAlchemy sessions
- broker clients

- provider payloads
 - infrastructure connection objects
- Widgets shall not call broker adapters directly.

State Representation

Operational state shall use explicit types.

Prefer:

```
```python
BrokerConnectionState.CONNECTED
MarketDataState.STALE
OrderStatus.ACKNOWLEDGED
```
```

Avoid loosely defined string states such as:

```
```python
"ok"
"active"
"done"
```
```

State transitions shall remain deterministic where business-critical.

Timestamps

API contracts containing time-dependent data shall preserve timestamp semantics.

Where relevant distinguish:

- market timestamp
- broker timestamp
- application timestamp

Timezone context shall remain explicit.

Do not expose ambiguous naive timestamps across critical interfaces.

Availability

Financial and trading data APIs shall preserve unavailable state.

Do not convert missing data silently to:

```
```python
0
0.0
Decimal("0")
```
```

when zero has valid business meaning.

Use explicit optional or availability-aware structures.

API Documentation

Public or cross-capability APIs shall document:

- purpose
- consumer
- parameters
- return value
- failure behaviour
- async behaviour where relevant
- timeout behaviour where relevant
- idempotency where relevant
- side effects

Usage examples should be provided for non-obvious interfaces.
Internal private methods do not require redundant documentation when behaviour is clear.

API Versioning

Internal Python interfaces evolve with the application.

Breaking internal changes require:

- consumer updates
- automated test updates
- documentation updates where the contract is documented

Formal API versioning is not required for internal interfaces unless independent consumers exist.

Future external APIs require an explicit versioning strategy before release.

Backward Compatibility

Backward compatibility shall be evaluated based on actual consumers.

Do not preserve obsolete internal interfaces solely for hypothetical future compatibility.

Public or independently consumed interfaces require controlled migration.

Compatibility requirements shall be explicit.

API Security

APIs shall:

- validate external input
- reject invalid requests explicitly
- avoid exposing sensitive information
- preserve least-privilege principles
- avoid logging secrets

Broker credentials shall never be exposed through application-facing API contracts.

Sensitive provider responses shall be normalized before presentation exposure where required.

Testing

Critical interfaces require automated tests.

Tests shall verify where relevant:

- expected behaviour
- edge cases
- invalid input
- failure translation
- timeout behaviour
- cancellation
- state translation
- duplicate submission protection
- retry behaviour
- unavailable data handling

Adapter contract tests should verify provider translation boundaries.

Tests shall not require LIVE trading.

API Evolution

APIs evolve with concrete product workflows.

Before extending an interface:

1. Identify the new consumer requirement.
2. Verify the existing interface responsibility.
3. Prefer extending the correct capability boundary.

4. Avoid unrelated operations on existing interfaces.
 5. Evaluate backward compatibility for real consumers.
 6. Update tests.
 7. Update documentation.
- Avoid generic APIs created for hypothetical future reuse.
-

Future External APIs

Potential future interfaces may include:

- Plugin APIs
- REST APIs
- WebSocket APIs

These interfaces are not part of the initial Trading Cockpit architecture.

Before introducing an external API define:

- product requirement
- consumer
- authentication
- authorization
- versioning
- compatibility policy
- rate limits where relevant
- transport error model
- security review

External API contracts shall not expose internal implementation models directly.

API Review Checklist

Before introducing or changing an API verify:

- product or technical requirement identified
 - consumer identified
 - owner identified
 - capability boundary identified
 - interface remains minimal
 - naming is explicit
 - parameters are strongly typed
 - return state is meaningful
 - Domain types remain technology-independent
 - provider models remain in Infrastructure
 - persistence models remain in Infrastructure
 - async boundary is justified
 - cancellation considered
 - timeout behaviour considered
 - failure behaviour defined
 - error translation defined where required
 - idempotency evaluated
 - retry behaviour evaluated
 - duplicate execution risk evaluated
 - unavailable data remains explicit
 - timestamp semantics remain explicit
 - security impact evaluated
 - automated tests added
 - documentation synchronized
-

Related Documents

- Product_Vision.md

- Product_Roadmap.md
- Project_Overview.md
- Architecture.md
- Domain_Model.md
- Infrastructure.md
- Technical_Specifications.md
- Widget_Catalog.md
- UI_Guidelines.md
- Coding_Standards.md
- Development_Guidelines.md
- Testing_Strategy.md
- AGENTS.md